

3-Day Crash Plan

One hour per day

Prep materials — Anthropic Fellows OA

August 2026

Contents

3-Day Crash Plan — 1 hour/day	1
Day 1 — Python syntax (60 min)	1
Day 2 — Object-oriented Python (60 min)	2
Day 3 — Domain + mock OA (60 min)	2
Skip entirely	2
Test-day protocol	3

3-Day Crash Plan — 1 hour/day

Day	Focus	Material	Output
1	Python syntax refresher	python_syntax_reference.md	Drills c1-c8 passing
2	Object-oriented Python	refresher_day2_oop.md	Drills c9-c10 passing; class-reading drill
3	Inference engine + mock OA	inference_engine_primer.md mock_oa/	Levels 1-2 minimum

Rule for all three days: **type everything, run everything**. Reading alone is worth close to nothing here.

Day 1 — Python syntax (60 min)

Work through `python_syntax_reference.md`. It's a complete sweep — sections are tagged **[OA]** (test-relevant) and **[skim]** (completeness only).

Suggested 60-minute path if you can't do all 20 sections:

Min	Sections	Why
0-10	2 (numbers), 3 (strings)	Division semantics, f-strings, ceiling division
10-25	4 (lists), 5 (tuples/sets), 6 (dicts)	The containers every simulation is built from

Min	Sections	Why
25-35	7 (control flow), 8 (comprehensions)	Loop and generator fluency
35-50	10 (sorting) , 11 (collections/heapq)	Highest-value 15 minutes in the plan
50-60	16 (copying/mutability), 20 (traps)	The bugs that eat 20 minutes mid-test

Then run `python day1_drills.py` and complete drills **c1-c8**. Anything that takes over 3 minutes marks a section worth rereading. Answers are in `day1_drills_answers.py`.

Sections 12 (files/IO), 15 (typing), 17 (iteration protocol) can wait — they rarely appear in this format.

Day 2 — Object-oriented Python (60 min)

Work through `refresher_day2_oop.md`; use section 14 of the syntax reference as the dense companion cheat sheet.

- **0-30 min: writing classes.** `__init__` and `self`, instance vs. class attributes, `@property` vs. method, dunder methods, dataclasses (the `field(default_factory=...)` rule, the unhashable-by-default rule), `super()` and overriding, `NotImplementedError` stubs.
- **30-40 min: drills c9 and c10** — the Task dataclass and the Pipeline class. Small on purpose: the goal is producing class scaffolding without pausing to recall syntax.
- **40-60 min: the reading drill.** Open `mock_oa/engine.py` and give yourself 8 timed minutes to write down what each class owns, which members are properties vs. methods, which state you're responsible for updating, and where the mutation risk is. Check yourself against the answers at the end of the Day 2 refresher.

That last drill is the actual exam skill. The Reddit complaints were almost entirely about people skipping it and coding against a misread spec.

Day 3 — Domain + mock OA (60 min)

- **0-15 min: the domain.** `inference_engine_primer.md`, sections 2-5 plus the vocabulary list in section 8. You need enough that *prefill*, *decode*, *KV cache*, *continuous batching*, *load balancing*, and *preemption* don't slow you down in a README. The simulation logic is what's being tested, not the ML — don't go deeper than the primer.
- **15-60 min: the mock, timed at 45 minutes.** `mock_oa/`:
 - First 8 minutes: **read only** — `README.md`, then `engine.py`, then `tests.py`. The tests are the real spec; the README contradicts the code in at least one place, deliberately.
 - Next 12: Level 1. Run `python tests.py` as soon as it's plausible.
 - Remaining 25: Level 2, then Level 3 if it's going well. Skip Level 4.

Afterwards read `SPOILERS.md` and answer one question: *where did the time actually go?* If you have any slack left over the next day, that answer tells you what to redo.

Skip entirely

Graphs, DP, advanced algorithms, LeetCode grinding, PyTorch, GPU/attention math. None of it appears in this format. Queues, heaps, dicts, sorting with explicit tie-breaks, and careful

simulation are the whole toolkit.

Test-day protocol

1. Read the README **and the visible tests** first — 10 minutes, no code.
2. Before implementing, write down the tick order and every tie-break rule.
3. Trust tests > class signatures > README prose when they conflict.
4. Get Level 1 green early; it validates the model everything else builds on.
5. Budget roughly 15/20/25/30 minutes per level. Abandon a stuck level and bank partial credit.
6. Expect a later level to invalidate an earlier assumption — keep the admission/selection policy swappable rather than welded into the loop.
7. Print one debug line per tick while developing; delete before submitting.
8. If panic hits: 60 seconds of slow breathing, then restate the current level's contract in one written sentence before touching the keyboard.

Sleep matters more than a fourth hour of prep. Being unhurried in the first ten minutes is the single highest-leverage thing you control.